

UIKit UI Dasturlash Bo'yicha Mukammal Qo'llanma (Zero to Hero)

Ushbu qo'llanma UIKit'da dasturlashni boshlayotgan, ayniqsa Flutter tajribasiga ega bo'lgan dasturchilar uchun mo'ljallangan. Unda sodda `frame` tushunchasidan boshlab, `Auto Layout`, barcha asosiy UI komponentlar, formalar, pickerlar, ekran navigatsiyasi va eng murakkab animatsiyalargacha bosqichma-bosqich yoritilgan.

Mundarija

- Mental Model: Flutter vs UIKit UI
- Frame-based Layout vs Auto Layout
- UIViewController va Ekranning Hayot Davri (Lifecycle)
- Asosiy UI Elementlar (Label, Button, Image, StackView)
- Dizayn va Styling (Burchaklar, Soyalar, Gradient, Blur)
- Input Widgetlar (TextField, TextView, Switch, Slider)
- Date & Time Picker (UIDatePicker)
- Rasm va Fayl Tanlash (Pickerlar)
- Navigatsiya va Ekran O'tishlari
- Ekranlararo Ma'lumot Olib O'tish (Data Passing)
- Animatsiyalar (Soddadan Murakkabgacha)

1. Mental Model: Flutter vs UIKit UI

Flutter deklarativ (Declarative) — siz holatga (state) qarab "UI qanday ko'rinishi kerak"ligini aytasiz (`build()` qayta ishlaydi).

UIKit esa imperativ (Imperative) — siz UI obyektini bir marta yaratasiz, ekranga qo'shasiz (`addSubview`) va keyinchalik uning xususiyatlarini

to'g'ridan-to'g'ri o'zgartirasiz (`label.text = "Salom"`).

Flutter Konsepsiyasi	UIKit Ekvivalenti	Izoh
Widget	UIView (yoki uning subklassi)	Ekranda chiziladig an barcha narsalar
Scaffold / Screen	UIViewController	Ekranni boshqaru vchi kontroller
Column / Row	UIStackView	Vertikal yoki gorizonta l tartiblash
Container / SizedBox	UIView	Bo'sh joy yoki fon qutisi
Text	UILabel	Matn chiqarish
ElevatedButt on	UIButton (Configuration bilan)	Tugma
Image.asset	UIImageView	Rasm
TextField	UITextField / UITextView	Matn kiritish

<code>ListView.builder</code>	<code>UITableView / UICollectionView</code>	Ro'yxat ko'rinishi
<code>Navigator.push</code>	<code>navigationController?.pushViewController</code>	Navigatsiya

2. Frame-based Layout vs Auto Layout

2.1 Frame Nima va Nega U Yetarli Emas?

`frame` — bu view'ning ota-onasiga nisbatan **aniq koordinatalari va o'lchami**:



SWIFT

```
myView.frame = CGRect(x: 20, y: 100, width: 200, height: 50)
```

MUHIM OGOHLANTIRISH

Nimaga faqat `frame` bilan ishlash tavsiya etilmaydi?

1. iPhone o'lchamlari juda ko'p: iPhone SE (kichik), iPhone 16 Pro Max (ulkan), iPad. Aniq piksellarda yozilgan `frame` bitta ekranda chiroyli ko'rinsa, boshqasida chetga chiqib ketadi yoki kichrayib qoladi.
2. Ekran aylanganda (Orientation) `frame`larni qaytadan hisoblab chiqish kerak bo'ladi.
3. Dynamic Type (foydalanuvchi shriftni kattalashtirganda) `frame` moslashmaydi.

2.2 Auto Layout — Zamonaviy Yondashuv

Auto Layout — bu qoidalarga (Constraints) asoslangan tizim. Siz "`x = 20 bo'lsin`" demaysiz, balki "**ekranning chap chetidan 20pt masofada tursin, eni ota-onasiga teng bo'lsin**" deysiz.

Eng Muhim Qoida:

Kodingizda Auto Layout ishlatmoqchi bo'lsangiz, **HAR DOIM** ushbu xususiyatni `false` qilishingiz shart:



SWIFT

```
myView.translatesAutoresizingMaskIntoConstraints = false
```

(Agar buni yozmasangiz, iOS avtomatik ravishda eski frame qoidalarini qo'llaydi va siz yozgan cheklovlar ishlamaydi).

NSLayoutAnchor Orqali Cheklovlar Berish:

SWIFT

```
let box = UIView()
box.backgroundColor = .systemBlue
box.translatesAutoresizingMaskIntoConstraints = false

view.addSubview(box) // AVVAL view'ga qo'shiladi, KEYIN
constraint beriladi!

NSLayoutConstraint.activate([
    // Yuqoridan Safe Area'ga nisbatan 20pt pastda
    box.topAnchor.constraint(equalTo:
view.safeAreaLayoutGuide.topAnchor, constant: 20),

    // Chapdan 16pt
    box.leadingAnchor.constraint(equalTo: view.leadingAnchor,
constant: 16),

    // O'ngdan 16pt (manfiy son bilan ichkariga suriladi)
    box.trailingAnchor.constraint(equalTo:
view.trailingAnchor, constant: -16),

    // Balandligi aniq 100pt
    box.heightAnchor.constraint(equalToConstant: 100)
])
```

Safe Area Nima?

`view.safeAreaLayoutGuide` — bu iPhone'ning tepasidagi "Notch / Dynamic Island" va pastidagi "Home Bar"ni hisobga olgan xavfsiz zona. Kontentingiz Dynamic Island ostida qolib ketmasligi uchun `view.topAnchor` emas, balki `view.safeAreaLayoutGuide.topAnchor` ga bog'laysiz.

3. UIViewController va Ekranning Hayot Davri (Lifecycle)

Har bir ekran — bu `UIViewController`. U o'zining hayot davriga ega:

init()

Klass xotiraga yuklanishi



loadView()

Ekraning asosiy view'si yaratilishi



viewDidLoad()

UI va bir martalik sozlamalar yuklanishi



viewWillAppear()

Ekran foydalanuvchiga ko'rinishidan oldin



viewDidLayoutSubviews()

Frame va Auto Layout to'liq hisoblanganda



viewDidAppear()

Ekran foydalanuvchiga to'liq ko'ringanda



viewWillDisappear()

Ekran yopilishni boshlaganda



viewDidDisappear()

Ekran to'liq yopilganda



deinit()

Ekran xotiradan tozalanishi

Amalda qaysi metodda nima qilinadi?



SWIFT

```
class MyViewController: UIViewController {  
  
    // 1. UI elementlarni xususiyat (property) sifatida e'lon  
qilamiz  
    private let titleLabel: UILabel = {  
        let label = UILabel()  
        label.text = "Asosiy Ekran"  
        label.font = .systemFont(ofSize: 24, weight: .bold)  
        label.translatesAutoresizingMaskIntoConstraints =  
false  
        return label  
    }()  
}
```

```

    override func viewDidLoad() {
        super.viewDidLoad()
        // Bu yerda bir martalik ishlar qilinadi:
        view.backgroundColor = .systemBackground
        setupUI()
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        // Ekran har safar ko'ringanda yangilanishi kerak
        bo'lgan ma'lumotlar
    }

    private func setupUI() {
        view.addSubview(titleLabel)

        NSLayoutConstraint.activate([
            titleLabel.centerXAnchor.constraint(equalTo:
view.centerXAnchor),
            titleLabel.centerYAnchor.constraint(equalTo:
view.centerYAnchor)
        ])
    }
}

```

4. Asosiy UI Elementlar

4.1 UILabel (Matnlar)



SWIFT

```

let label = UILabel()
label.translatesAutoresizingMaskIntoConstraints = false
label.text = "Salom, iOS!"
label.textColor = .label // Dark/Light
mode'ga avtomatik moslashuvchi rang
label.font = UIFont.systemFont(ofSize: 18, weight: .semibold)
label.textAlignment = .center
label.numberOfLines = 0 // 0 = cheksiz
qator (Flutter'dagi kabi matn sig'sa keyingi qatorga o'tadi)
label.lineBreakMode = .byWordWrapping

```

4.2 UIButton (Tugmalar — Zamonaviy iOS 15+ Yondashuv)

Eski `button.setTitle` o'rniga zamonaviy `UIButton.Configuration` ishlatiladi:

SWIFT

```
var config =
UIButton.Configuration.filled() // .tinted(), .gray(), .plain(
) mavjud
config.title = "Davom etish"
config.image = UIImage(systemName: "arrow.right")
config.imagePlacement = .trailing // Rasm matndan
o'ngda
config.imagePadding = 8
config.baseBackgroundColor = .systemBlue
config.baseForegroundColor = .white
config.cornerStyle = .capsule // Oval tugma
(.medium, .large ham bor)

let button = UIButton(configuration: config)
button.translatesAutoresizingMaskIntoConstraints = false

// Bosish hodisasini tinglash (Action):
button.addTarget(self, action: #selector(buttonTapped),
for: .touchUpInside)

@objc private func buttonTapped() {
    print("Tugma bosildi!")
}
```

4.3 UIImageView (Rasmlar)

SWIFT

```
let imageView = UIImageView()
imageView.translatesAutoresizingMaskIntoConstraints = false
imageView.image = UIImage(systemName: "star.fill") // SF
Symbol
// Yoki Assets'dagi rasm: UIImage(named: "my_photo")
imageView.contentMode = .scaleAspectFill //
Flutter'dagi BoxFit.cover
imageView.clipsToBounds = true //
Chegaradan chiqqan qismini qirgib tashlash
imageView.tintColor = .systemYellow
```

4.4 UIStackView (Flutter'dagi `Column` va `Row`)

StackView bir nechta elementlarni avtomatik qator yoki ustun qilib terish uchun eng qulay vosita:



```
SWIFT
let stackView = UIStackView(arrangedSubviews: [titleLabel,
imageView, button])
stackView.translatesAutoresizingMaskIntoConstraints = false
stackView.axis = .vertical // .horizontal bo'lsa
Row, .vertical bo'lsa Column
stackView.spacing = 16 // Elementlar orasidagi
masofa
stackView.alignment = .fill // Elementlar kengligi
(.center, .leading, .trailing)
stackView.distribution = .fill // O'lchamlarni
taqsimlash

view.addSubview(stackView)

NSLayoutConstraint.activate([
    stackView.topAnchor.constraint(equalTo:
view.safeAreaLayoutGuide.topAnchor, constant: 20),
    stackView.leadingAnchor.constraint(equalTo:
view.safeAreaLayoutGuide.leadingAnchor, constant: 16),
    stackView.trailingAnchor.constraint(equalTo:
view.safeAreaLayoutGuide.trailingAnchor, constant: -16)
])
```

5. Dizayn va Styling

Har bir `UIView` orqasida uning grafik qatlami bo'lgan `CALayer` yotadi. Burchaklar, soyalar va chegaralar `layer` orqali sozlanadi.

5.1 Burchaklarni Yumaloqlash (Corner Radius)



```
SWIFT
cardView.layer.cornerRadius = 16
// Agar faqat ma'lum burchaklarni yumaloqlamoqchi bo'lsangiz:
cardView.layer.maskedCorners =
```

```
[.layerMinXMinYCorner, .layerMaxXMinYCorner] // Faqat yuqori burchaklar
```

5.2 Chegaralar (Border)



SWIFT

```
cardView.layer.borderWidth = 1.5
cardView.layer.borderColor = UIColor.systemGray4.cgColor //
Diqqat: .cgColor kutiladi
```

5.3 Soylar (Drop Shadow)



SWIFT

```
cardView.layer.shadowColor = UIColor.black.cgColor
cardView.layer.shadowOffset = CGSize(width: 0, height: 4) // X
va Y yo'nalishidagi siljish
cardView.layer.shadowOpacity = 0.15 //
Shaffoflik (0.0 dan 1.0 gacha)
cardView.layer.shadowRadius = 8 //
Xiralashish radiusi
cardView.layer.masksToBounds = false //
true bo'lsa soya qirgilib ketadi!
```

FOYDALI MASLAHAT (TIP)

Soya va CornerRadius birga ishlatilganda:

Agar `clipsToBounds = true` qilsangiz, soya ko'rinmay qoladi. Yechim:

Tashqi ota view'ga soya beriladi (`masksToBounds = false`), ichki subview'ga esa rasm va `cornerRadius` beriladi (`clipsToBounds = true`).

5.4 Gradient Fon (CAGradientLayer)



SWIFT

```
let gradientLayer = CAGradientLayer()
gradientLayer.colors = [
    UIColor.systemPurple.cgColor,
    UIColor.systemBlue.cgColor
]
gradientLayer.startPoint = CGPoint(x: 0, y: 0) // Chap yuqori
gradientLayer.endPoint = CGPoint(x: 1, y: 1) // O'ng pastki
gradientLayer.frame = view.bounds

view.layer.insertSublayer(gradientLayer, at: 0) // Eng ostki
```

qatlarga joylash

5.5 Xiralashtirish Effekti (Glassmorphism / Blur)

SWIFT

```
let blurEffect =  
UIBlurEffect(style: .systemUltraThinMaterial) // Zamonaviy  
oyna effekti  
let blurView = UIVisualEffectView(effect: blurEffect)  
blurView.translatesAutoresizingMaskIntoConstraints = false  
blurView.layer.cornerRadius = 20  
blurView.clipsToBounds = true  
  
view.addSubview(blurView)  
// Constraintlar beriladi...
```

6. Input Widgetlar

6.1 UITextField (Bir qatorli matn kiritish)

SWIFT

```
let textField = UITextField()  
textField.translatesAutoresizingMaskIntoConstraints = false  
textField.placeholder = "Telefon raqamingiz yoki emailingiz"  
textField.borderStyle = .roundedRect  
textField.keyboardType  
= .emailAddress // .numberPad, .phonePad, .default  
textField.isSecureTextEntry = false // true  
bo'lsa parol ko'rinishida yashiradi (dots)  
textField.clearButtonMode = .whileEditing // Matnni  
tozalovchi (x) tugmasi  
textField.returnKeyType = .done //  
Klaviaturadagi 'Go', 'Search', 'Done' tugmasi  
  
// Hodisalarni tinglash (Delegate orqali):  
textField.delegate = self
```

Klaviaturani yopish uchun ekranning istalgan joyi bosilganda:

SWIFT

```
override func touchesBegan(_ touches: Set<UITouch>, with
event: UIEvent?) {
    view.endEditing(true)
}
```

6.2 Zamonaviy Klaviatura Animatsiyasi (iOS 15+ keyboardLayoutGuide)

Eskidan klaviatura ochilganda TextField'ni to'sib qo'ymaslik uchun ancha kod yozilar edi. Endi bu 1 qatorda hal bo'ladi:

SWIFT

```
// Tugmani yoki inputni klaviatura balandligiga qarab
avtomatik ko'tarish:
bottomConstraint = sendButton.bottomAnchor.constraint(equalTo:
view.keyboardLayoutGuide.topAnchor, constant: -16)
bottomConstraint.isActive = true
```

6.3 UISwitch va UISegmentedControl

SWIFT

```
// Switch (Toggle)
let toggle = UISwitch()
toggle.isOn = true
toggle.addTarget(self, action: #selector(switchChanged(_:)),
for: .valueChanged)

@objc func switchChanged(_ sender: UISwitch) {
    print("Switch holati: \(sender.isOn)")
}

// Segmented Control (Tabs)
let segment = UISegmentedControl(items: ["Barchasi", "Yangi",
"Arxiv"])
segment.selectedSegmentIndex = 0
segment.addTarget(self, action: #selector(segmentChanged(_:)),
for: .valueChanged)

@objc func segmentChanged(_ sender: UISegmentedControl) {
    print("Tanlangan indeks: \(sender.selectedSegmentIndex)")
}
```

7. Date & Time Picker

Apple zamonaviy iOS'da 3 xil ko'rinishdagi `UIDatePicker`ni taqdim etadi:

- `.inline` — to'liq ochiq taqvim (Calendar sheet).
- `.compact` — kichik, bosilganda ochiladigan badge.
- `.wheels` — klassik aylanuvchi g'ildirak.

SWIFT

```
let datePicker = UIDatePicker()
datePicker.translatesAutoresizingMaskIntoConstraints = false
datePicker.datePickerMode
= .dateAndTime // .date, .time, .countDownTimer
datePicker.preferredDatePickerStyle
= .inline // .compact, .wheels
datePicker.minimumDate = Date() // Faqat
bugundan keyingi kunlar
datePicker.addTarget(self, action: #selector(dateChanged(_)),
for: .valueChanged)

view.addSubview(datePicker)

@objc func dateChanged(_ sender: UIDatePicker) {
    let formatter = DateFormatter()
    formatter.dateFormat = "dd.MM.yyyy, HH:mm"
    let dateString = formatter.string(from: sender.date)
    print("Tanlangan sana: \(dateString)")
}
```

8. Rasm va Fayl Tanlash

8.1 UIImagePickerController (Zamonaviy Galereya Pickeri)

Foydalanuvchidan hech qanday maxsus permission (ruxsat) so'ramaydi va eng xavfsiz hisoblanadi.

SWIFT

```

import PhotosUI

func openGallery() {
    var config = PHPickerConfiguration()
    config.selectionLimit = 1 // 1 ta yoki
    ko'p rasm tanlash
    config.filter = .images // .videos
    yoki .any(of: [.images, .livePhotos])

    let picker = PHPickerViewController(configuration: config)
    picker.delegate = self
    present(picker, animated: true)
}

// Delegate:
extension MyViewController: PHPickerViewControllerDelegate {
    func picker(_ picker: PHPickerViewController,
    didFinishPicking results: [PHPickerResult]) {
        picker.dismiss(animated: true)

        guard let provider = results.first?.itemProvider,
        provider.canLoadObject(ofClass: UIImage.self)
    else { return }

        provider.loadObject(ofClass: UIImage.self) { [weak
self] image, error in
            guard let selectedImage = image as? UIImage else
{ return }
            DispatchQueue.main.async {
                self?.myImageView.image = selectedImage
            }
        }
    }
}

```

8.2 UIDocumentPickerViewController (PDF / Fayl Tanlash)

SWIFT

```

import UniformTypeIdentifiers

func openFilePicker() {
    let supportedTypes: [UTType] = [.pdf, .text, .image]
    let documentPicker =
UIDocumentPickerViewController(forOpeningContentTypes:

```

```
supportedTypes, asCopy: true)
    documentPicker.delegate = self
    documentPicker.allowsMultipleSelection = false
    present(documentPicker, animated: true)
}

extension MyViewController: UIDocumentPickerDelegate {
    func documentPicker(_ controller:
UIDocumentPickerViewController, didPickDocumentsAt urls:
[URL]) {
        guard let selectedURL = urls.first else { return }
        print("Tanlangan fayl yo'li: \(selectedURL.path)")
    }
}
```

9. Navigatsiya va Ekran O'tishlari

iOS'da asosan 2 xil ekran ochish usuli bor: **Push/Pop** (Stack orqali) va **Present/Dismiss** (Modal orqali).

9.1 Push & Pop (UINavigationController)

Gorizontal siljib keladigan sahifalar uchun:



SWIFT

```
// 1. Yangi ekranni ochish (Push)
let detailsVC = DetailsViewController()
navigationController?.pushViewController(detailsVC, animated:
true)

// 2. Orqaga qaytish (Pop)
navigationController?.popViewController(animated: true)

// 3. Eng boshiga qaytish (Pop to Root)
navigationController?.popToRootViewController(animated: true)
```

9.2 Modal Oyna (Present & Dismiss)

Pastdan yuqoriga qalqib chiquvchi oyna:



SWIFT

```
let modalVC = FilterViewController()
// To'liq ekran qilish uchun: modalVC.modalPresentationStyle
= .fullScreen
present(modalVC, animated: true)

// O'zini yopish (ichki ekrandan):
dismiss(animated: true)
```

9.3 Zamonaviy Bottom Sheet (iOS 15+ Detents)

Yarim ekran yoki to'liq ekran bo'lib ochiladigan Telegram yoki Apple Maps uslubidagi oyna:



```
SWIFT
let sheetVC = BottomMenuViewController()
if let sheet = sheetVC.sheetPresentationController {
    sheet.detents = [.medium(), .large()] // Yarim va
to'liq ochilish o'lchamlari
    sheet.prefersGrabberVisible = true // Yuqoridagi
tortish chiziqchasi (grabber)
    sheet.preferredCornerRadius = 24
}
present(sheetVC, animated: true)
```

10. Ekranlararo Ma'lumot Olib O'tish

1-usul: Oldinga ma'lumot uzatish (Initializer orqali - Eng toza usul)



```
SWIFT
// 2-ekran:
class DetailViewController: UIViewController {
    private let productId: String

    init(productId: String) {
        self.productId = productId
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError() }
}
```

```
// 1-ekrandan chaqirish:  
let detailVC = DetailViewController(productId: "PR-12345")  
navigationController?.pushViewController(detailVC, animated:  
true)
```

2-usul: Orqaga ma'lumot qaytarish (Closure / Callback orqali)

Flutter'dagi `Navigator.pop(context, result)` kabi eng oson usul:

SWIFT

```
// 2-ekran (FilterViewController):  
class FilterViewController: UIViewController {  
    var onFilterSelected: ((String) -> Void)?  
  
    @objc func applyTapped() {  
        onFilterSelected?("Electronics")  
        dismiss(animated: true)  
    }  
}  
  
// 1-ekran (chaqiruvchi):  
let filterVC = FilterViewController()  
filterVC.onFilterSelected = { selectedCategory in  
    print("Tanlangan kategoriya: \(selectedCategory)")  
}  
present(filterVC, animated: true)
```

3-usul: Delegate Pattern (Klassik iOS usuli)

SWIFT

```
// Shartnoma (Protocol):  
protocol EditProfileDelegate: AnyObject {  
    func didUpdateName(_ newName: String)  
}  
  
// 2-ekran:  
class EditProfileViewController: UIViewController {  
    weak var delegate: EditProfileDelegate?  
  
    @objc func saveTapped() {  
        delegate?.didUpdateName("Ali Valiyev")  
    }  
}
```

```

        dismiss(animated: true)
    }
}

// 1-ekran:
class ProfileViewController: UIViewController,
EditProfileDelegate {
    func openEdit() {
        let editVC = EditProfileViewController()
        editVC.delegate = self
        present(editVC, animated: true)
    }

    func didUpdateName(_ newName: String) {
        nameLabel.text = newName
    }
}

```

11. Animatsiyalar (Soddadan Murakkabgacha)

1-Daraja: Standart UIView Animatsiyasi

O'zgartiriladigan xususiyatlar: `alpha`, `backgroundColor`, `transform`, `frame`.



SWIFT

```

UIView.animate(withDuration: 0.3) {
    self.myView.alpha = 0.0 // Yo'q
    bo'lish (Fade out)
    self.myView.transform = CGAffineTransform(scaleX: 0.8, y:
0.8) // Kichrayish
}

```

2-Daraja: Prujinali Animatsiya (Spring Animation)

Haqiqiy fizik prujina effekti:



SWIFT

```

UIView.animate(
    withDuration: 0.6,

```

```

        delay: 0,
        usingSpringWithDamping: 0.6, // 0.1 bo'lsa juda tebranadi,
1.0 bo'lsa umuman tebranmaydi
        initialSpringVelocity: 0.5,
        options: [.curveEaseInOut]
    ) {
        self.myCard.transform = .identity // Asl holatiga
qaytarish
    }
}

```

3-Daraja: Auto Layout Cheklovlarini Animatsiya Qilish

Auto Layout'da view'ning **frame**ini emas, balki **constraint**'ning **constant**ini o'zgartirasiz:



SWIFT

```

// Masalan balandlikni 50 dan 200 ga o'zgartiramiz:
self.cardHeightConstraint.constant = 200

UIView.animate(withDuration: 0.4) {
    self.view.layoutIfNeeded() // Barcha o'zgarishlarni
animatsiya bilan chizish buyrug'i
}

```

4-Daraja: Murakkab Core Animation (Masalan: Xatolikda Tebranish — Shake Effect)

Matn noto'g'ri kiritilganda TextField tebranib xatolik berishi:



SWIFT

```

extension UIView {
    func shake() {
        let animation = CAKeyframeAnimation(keyPath:
"position.x")
        animation.values = [0, 10, -10, 10, -5, 5, 0]
        animation.keyTimes = [0, 0.16, 0.33, 0.5, 0.66, 0.83,
1]
        animation.duration = 0.4
        animation.isAdditive = true
        self.layer.add(animation, forKey: "shake")
    }
}

```

```
// Ishlatish:  
passwordTextField.shake()
```

Xulosa va O'rganish Bo'yicha Tavsiyalar

- **Auto Layout'ga o'tish:** Hech qachon piksellarni hisoblab `frame` bermang. Doim `translateAutoresizingMaskIntoConstraints = false` qiling va `NSLayoutAnchor` dan foydalaning.
- **UIStackView'dan maksimal foydalaning:** Flutter'dagi `Column` va `Row` odatlarini `UIStackView` orqali tezda UIKit'da qo'llashingiz mumkin.
- **Kichik tajribalar qiling:** Har bir mavzuni (masalan, avval bitta chiroyli Card yaratish, keyin DatePicker ulash, so'ng Shake animatsiyasini sinash) alohida qismlarda test qilib boring.